자동완성에서 에이전트까지

프롬프트로 돌아보는 개발자와 AI

저는요

- C/C++ 기반 데이터베이스 접근제어 보안 소프트웨어
- 메타버스 서비스
- 블록체인 상 탈-중앙 NFT 거래 엔진
- 교육 플랫폼 – LLM 파이프라인
- 교육업계 비대면 결제 플랫폼 – PG, 온라인 VAN

- 김영민 (30대 남성, 무직) – (BE기반) 서비스 엔지니어

목차

0장 **제목 & 소개**	
1장 **코드 써주세요**	구현을 묻고, AI가 쓰는 시대
2장 **이 안에서** **구현해주세요**	AI가 코드를 쓸 때 울타리를 잘 치는 '**프롬프트 엔지니어링**'
3장 **이 문제를** **해결해주세요**	에이전트가 직접 프로젝트 구성, 탐색, 계획을 통해 문제 해결
4장 **결론**	우리는 뭐하는 사람일까요

1장 코드 써주세요

코드 자동완성의 시대 (2021)

- AI = 기계 학습(Machine learning) 기반 다음 코드 예측

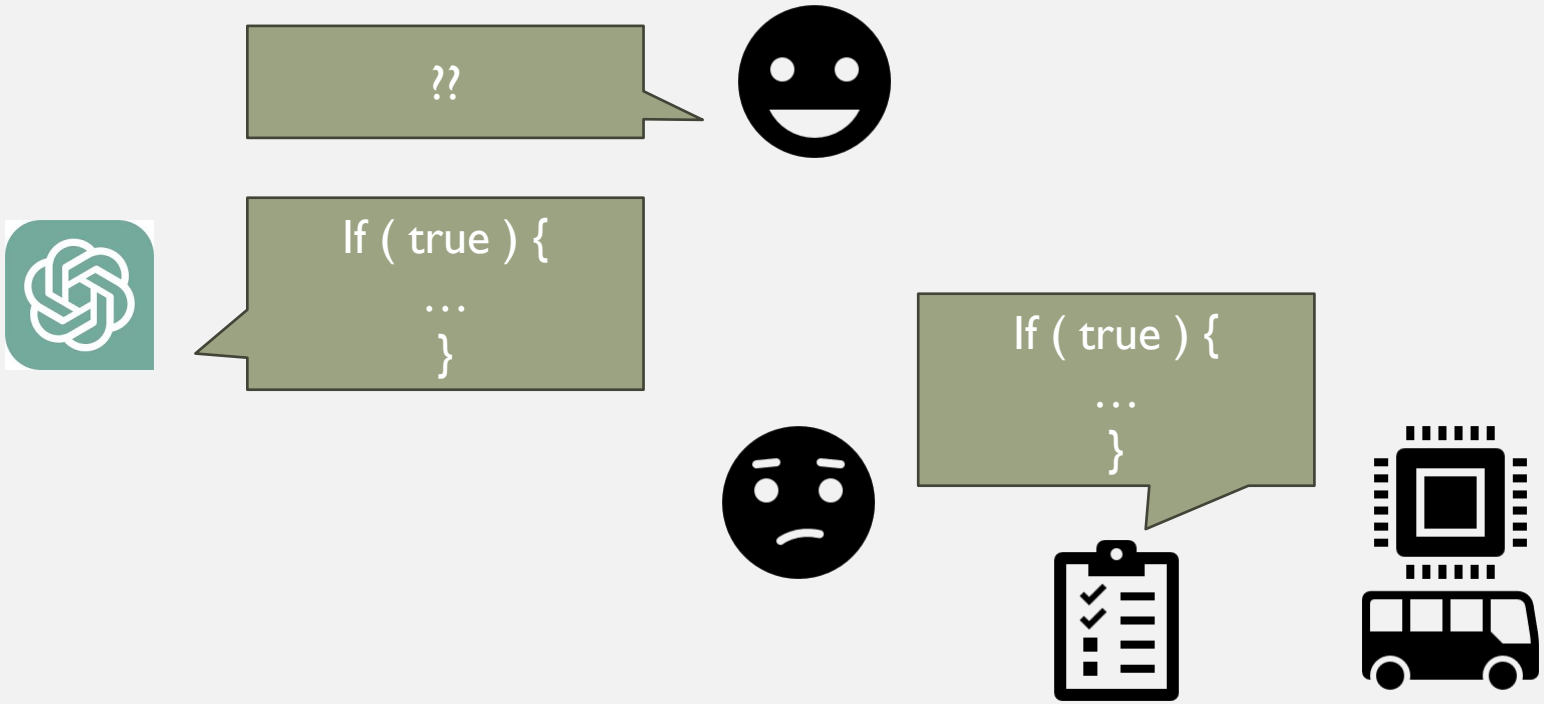
Ex) 고인물 개발자의 코드 템플릿

반복 패턴 보완, 시간 단축

Kite
ML 기반 코드 예측 + IDE 플러그인

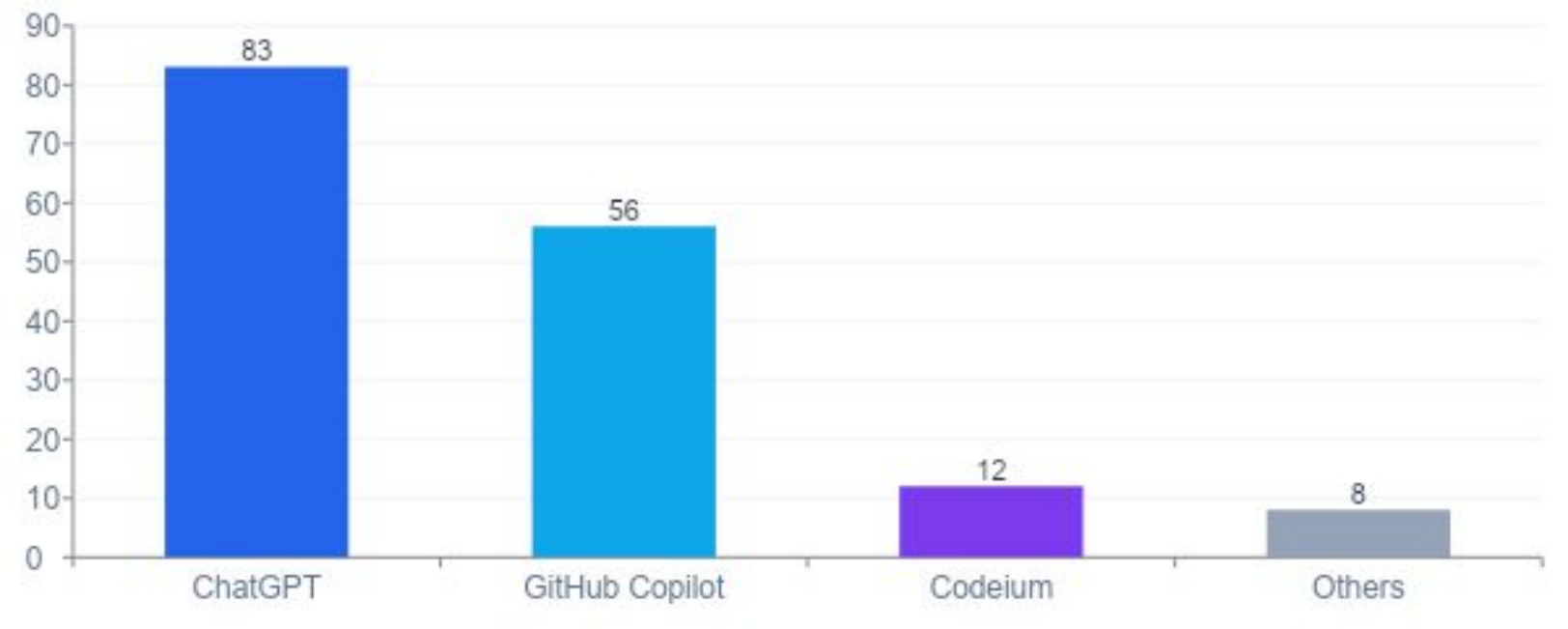
Tabnine
통계·ML 기반 코드 자동완성

GPT-3.5의 등장, 사람은 클립보드 서버 (2023)



2023년 AI 도구별 사용률

* 중복 응답 가능, 46,208명



단, 정확도 면에서 매우 신뢰한다 (5/5) 이라고 답한 비율은 단 3%

https://survey.stackoverflow.co/2024/technology

클립보드 시대의 한계와 새로운 워크플로우

- 모델의 환각

  run(onSuccess, onFail){ … }

  …
  const result = run(1024, () => {…}, false);

- 워크플로우 문제

  물어보고, 붙여넣고, 빌드하고, 실행하고, 테스트하고, 틀렸다고 혼내고, 붙여넣고, 빌드하고, 실행하고, 테스트하고, 틀렸다고 혼내고, 산책 한번 나갔다가, 돌아와서, 물어보고, 붙여넣고, 빌드하고, 실행하고, 실행하려다 틀린거 발견하고, 혼내고, 물어보고, 붙여넣고, 빌드하고, 실행하고, 고쳤다!, 아니네?, 어디부터 했더라

  원래 이렇게 했던거같은데..?

2장 이 안에서 구현해주세요

코드 – 프로젝트가 프롬프트 (2024)

- 주석
- 시그니처 (파라미터 or 반환 형식)
- ENUM

구현 방향 예측, 자동완성 정확도 상승

GitHub Copilot

Copilot Chat
IDE 내부 대화형 코드 수정

CURSOR

Cursor
코드베이스 인덱싱 + @파일 참조

프롬프트의 구조화

요구사항 – 무엇을 만들어야 하는지

제약사항 – 무엇을 하면 안되는지

출력형식 – 어떤 형태(패턴)으로 결과를 내어야 하는가

예시코드 – **BestCase** 레퍼런스

가드레일?

묵직한 프롬프트 = 비용

- 결제와 구독 시스템에 라이선스 시스템을 연결하기 위해 결제는 이 서비스의 이 엔티티의 캡슐화를 존중하고 구독시스템은 여기 나온 데이터를 믿으면 되는데 캐시 레이어를 사용 할 수 있도록 경로를 먼저 조사하되 @subscribe.service.ts 파일을 확인 후 작성하고 라이선스는 상품이 가변적일 수 있으니 하드코딩 하지 않도록 하고 모르면 물어보고 @purchase.entity.ts 파일은 읽기 전용 쓰기 전용이 있을 수 있으니 확인하고 작업이 끝나면 @purchase.entity.spec.ts 파일의 테스트가 여전히 유효한지 확인하고 서비스 계층의 코드가 controller로 나가면 안됩니다. @auth.controller.ts 파일을 참고해서 인증 절차 등도 고려해주세요

이걸 매번 사람이 써야한다고..?

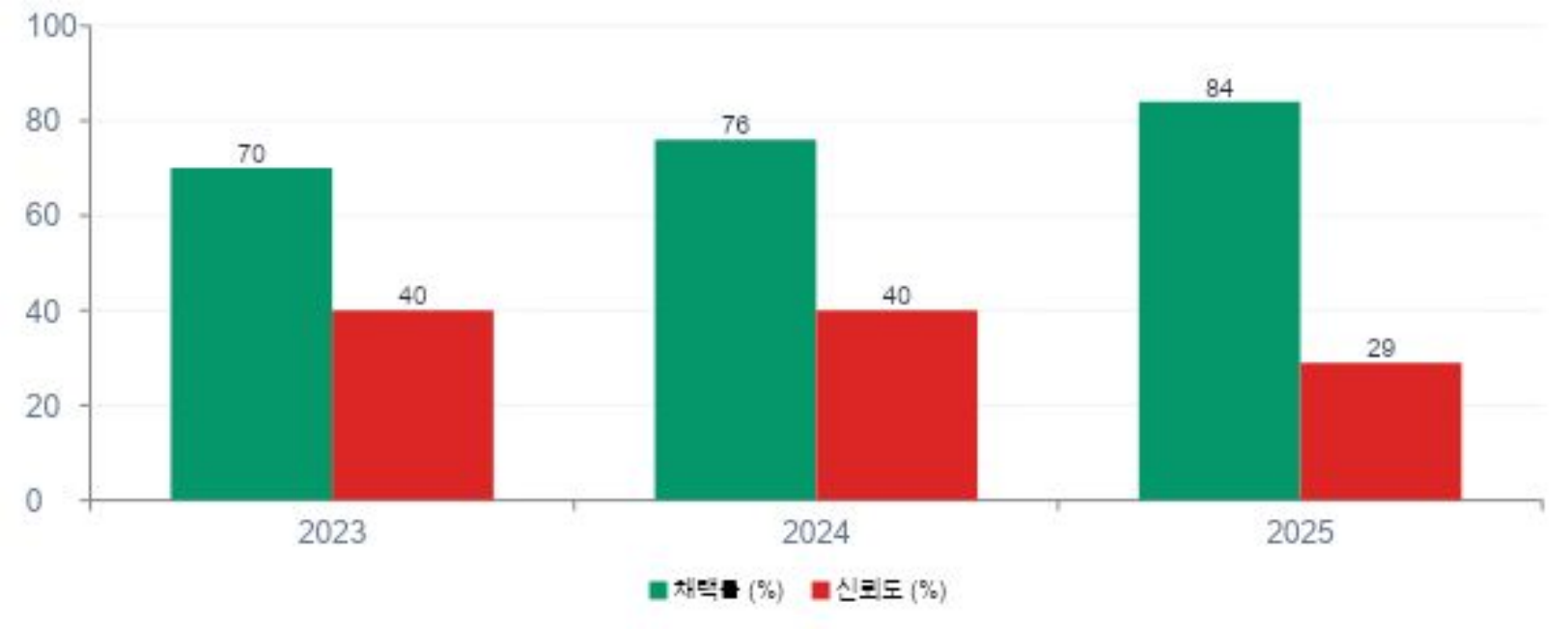
규칙 파일의 등장 (2024)

- .cursorrules
- CLAUDE.md
- AGENTS.md

- 엔티티의 캡슐화를 존중
- 구독시스템은 수정 전 /src/service/subscribe.service 를 참고
- 계층 구조를 따르지 않으면 캐시 불일치 문제가 생길 수 있음
- 몇몇 엔티티는 읽기 전용과 쓰기 전용 구분되어있음
- 영향에 따라 테스트 코드가 유효한지 확인할 것
- controller에서 비즈니스 로직을 구현하지 않도록 src/controller/user.controller.ts 파일을 확인
- 인증기능 맥락이 필요한경우 src/controller/auth.controller.ts 파일과 src/middleware.ts 참고

결제와 구독 시스템에 라이선스 시스템을 연결합니다

AI가 작성한 코드의 채택률과 신뢰도



3장 이 문제를 해결해주세요

GPT-4o

Claude 3.5 Sonnet

ANTHROP\C

Claude 3.7 Sonnet

코드 에이전트의 대중화 (2025)

- Agent Mode 가 등장
- Cursor, Github Copilot의 Agent Mode
- Claude Code 가 구독 사용자에게 개방

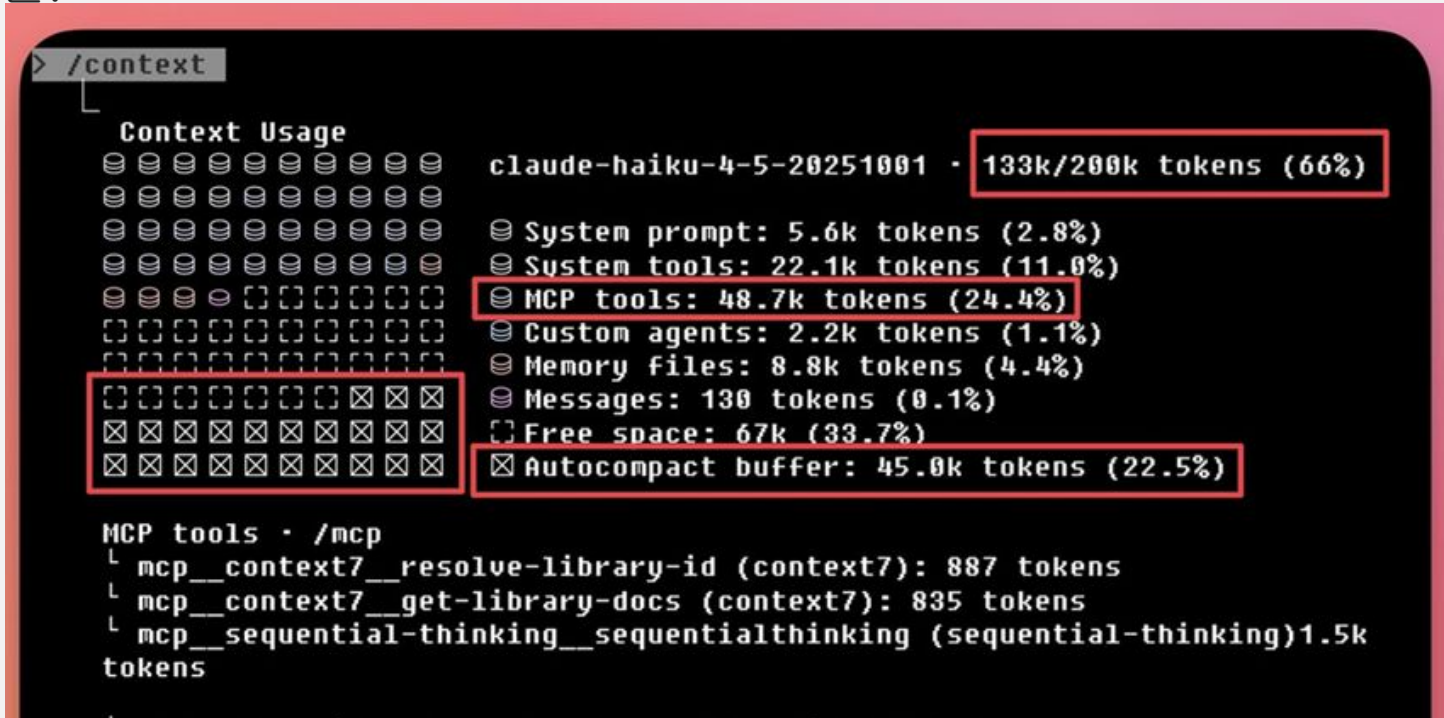
‘질문’ 을 잘 하는것 <> ‘실행’ 을 잘 하는것

문맥 기반으로 좋은 실행을 하는 것

MCP –에이전트의 HTTP급 범용성 (2024)

- M x N 문제
  저는 Cursor, Claude Code, Cline, Copilot 를 쓰고 도구는 10개를 씁니다
  구현해야 하는 연동 코드 수는?

- 표준 연결, 조합
- 브라우저, 생각 정리 등



스킬과 서브에이전트 – 에이전트 팀의 시대 (2025)

- 스킬 – 매뉴얼 인덱스 폴더, 스크립트 패키지
- 서브 에이전트 – 매뉴얼 읽어서 이거 해와

- 계획 – 탐색 – 구현 – 테스트

  개발자의 워크플로우가 그대로 구현됨

프롬프트 중심의 변화 – 구현 VS 문제

AS IS

- 이 컨벤션으로 작성해줘
- 이 형식을 참고해서 구조를 작성해줘
- 이렇게는 하면 안돼

TO-BE

- 이 문제를 해결해줘
- 이 리스크를 검토해줘
- 이 유즈케이스를 만족할 수 있어야 해

4장 결론

주니어와 시니어의 AI 사용

- AI를 잘 써서 코드를 잘 쓴다
    - 비가역적인 위협을 인지한다
    - 프로젝트의 설계 의도를 안다
    - 어떤 문맥이 참고되어야 하는지 안다
    - 자신 영역 외의 워크플로우를 안다
    - 어떻게 하면 실패하는지 안다

우리는 무엇을 하는 사람인가요

- 요구사항 협의 및 범위 조율
- 기술 방향 논의 및 의사결정
- 타 팀 의존성 조율
- 이슈 상황 보고
- API 명세 작성 및 유지
- 기술 설계 문서 (TDD, ADR)
- 온보딩 가이드
- 장애 사후 보고서 (Post-mortem)
- 의사결정 근거 기록

- 기능 개발 및 코딩
- 버그 디버깅 및 수정
- 코드 리뷰
- 테스트 작성
- 시스템 설계 및 아키텍처
- 배포 및 CI/CD 관리
- 성능 최적화 및 리팩터링
- 문서화
- 요구사항 분석 및 기획 협업
- 신기술 리서치 및 학습

- 기술 부채 식별 및 우선순위화
- 보안 취약점 점검
- 장애 대응 및 롤백 계획
- 의존성 버전 관리
- SPOF 파악 및 이중화 검토
- 모니터링 및 알람 설정

"에이전트 엔지니어링은 시니어에게 불균형적으

Addy Osmani — "Agentic Engineering "
(https://addyosmani.com/blog/agentic-engineering)

AI가 주는 이점의 본질

- 행동 (Act) 를 빠르게
- 행동의 문맥을 판단하기

- 개발이 쉽다보다 내가 생각하고 계획한것에 속도를 붙이는것
- 총알 – 예광탄, 탄도학
- 대륙간 탄도미사일 – 정밀 모니터링, 피드백

- 개발에만 한정되었을까?

Q&A

절찬리

구직중입니다

감사합니다

agomdole@gmail.com

https://resume.metts.today